

W. Shakespeare

Categories and functors:
much ado about nothing?

Bachelor thesis

26 April 1564

Thesis supervisors: prof.dr. G.A. Bredero
dr. J. van den Vondel



Leiden University
Mathematical Institute

Contents

1	Introduction	3
1.1	Assumptions	3
2	Relevant mathematics	4
2.1	Voronoi	4
2.2	Neighborhood search algorithms	4
2.2.1	Tabu search	5
2.2.2	Simulated annealing	5
3	General model	6
3.1	SAMP	6
3.2	Network structure	6
3.3	Transit Assignment	7
3.4	Costs	7
3.5	Access	8
3.6	The model	9
3.7	General algorithm	9
4	Applied model	10
4.1	Chicago	10
4.2	Leiden	10
4.2.1	Transit system	11
4.2.2	Facility and Community nodes	11
4.2.3	Chosen parameters	12
4.3	Explanation of experiment	13
4.4	Results	13
4.5	Discussion	13
A	Pseudocode	14
B	Data used	17
B.1	Busschedule	17
B.2	Busstops	17
B.3	General Practitioners	17
B.4	Communities	17
B.5	Travel data	17

Chapter 1

Introduction

1.1 Assumptions

The following things are assumed to be true

1. The group of people that the model is made for is very small compared to daily travelers, therefore this small group will have no effect on overcrowdedness or other usage related metrics.

Chapter 2

Relevant mathematics

2.1 Voronoi

Following the definition in [Møl94], a voronoi diagram is given by a set $\Phi = \{x_i\}$ of weightpoints in a metric space X . Each weightpoint generates a region

$$C(x_i|\Phi) = \{y \in X : d(x_i, y) \leq d(x_j, y) \quad \forall x_j \in \Phi\}$$

where d denotes some distance function. It follows that each voronoi region $C(x_i|\Phi)$ contains all points that have x_i as nearest weightpoint.

Applications of voronoi diagrams can be found in many fields. Consider for example the placement of automated external defibrillator (AED) boxes in a city. An even spread of these boxes would theoretically save the most people. The positions of these boxes could be used as weightpoints, with the euclidian distance in $\mathbb{R}^2$. This results in a voronoi diagram where smaller regions represent boxes that are relatively close to each other, while larger regions represent boxes that are far from others. This information could be used to move or add boxes, and thus getting a more even distribution.

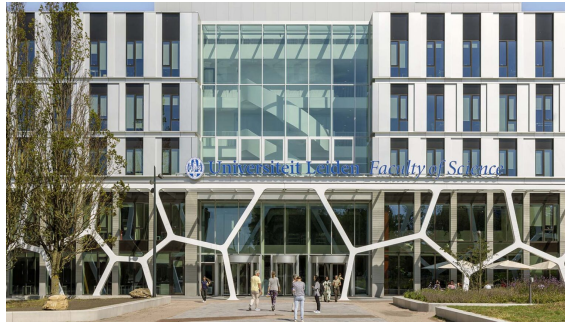


Figure 2.1: Entrance of Gorlaeus building of Leiden University, with voronoi diagram as decoration. Picture taken from [Swe25].

2.2 Neighborhood search algorithms

When an optimization problem has a large but finite solution space, a method of traversing possible solutions is a neighborhood search (NS) procedure. Given a solution space S , a neighborhood function $N : S \rightarrow \mathcal{P}$ and a feasible solution $s \in S$, a neighborhood $N(s)$ of s can be found. This contains all feasible solutions that can be reached from s after doing one *move*.

General NS algorithms are heuristic approaches and use a starting solution s^* and try to find a solution in $N(s^*)$ that has a more optimal value than s^* . If such solution s exists, a new iteration is done where a more optimal solution is sought in $N(s)$. This iteration loops until a solution does not have a more optimal solution in its neighborhood.

NS algorithms have no method of leaving local optima, and thus get stuck often. To prevent this behaviour, many methods have been developed. We discuss two of those methods, both relying on making suboptimal moves. Just like NS, these are heuristic approaches to optimization problems.

2.2.1 Tabu search

Tabu Search (TS) is a NS algorithm that uses *Tabu lists* to escape local maxima. Visited solutions or executed moves are stored in *short-term memory* (STM) and can't be revisited or redone for a small number of iterations, this is to prevent cycling. The length of time that solutions/moves are stored in STM is dependent on *Tabu tenures*. *Long-term memory* (LTM) is used to store solutions with a good objective value, so they, and their neighbors, can be revisited after the algorithm becomes trapped.

Tenures can be adjusted dynamically while running the algorithm. This is to allow a greater focus on *diversification*, the exploring of many different solutions, or on *intensification*, the exploiting of good solutions.

2.2.2 Simulated annealing

Simulated Annealing (SA) is named after a technique in metallurgy where a material is heated and cooled in a controlled fashion, resulting in a change in physical properties. SA uses the same iteration as a basic NS algorithm, comparing different neighbors to choose a good next move. When choosing a next step, a temperature parameter T is used to randomly choose a worse option. This parameter decreases while running, resulting in erratic moves in the beginning and more controlled moves towards the end of running.

Chapter 3

General model

This section summarizes the general model and algorithm developed by Rumpf and Kaul in [RK21]. As the model and algorithm is made by them, a more detailed explanation can be found in their paper. In this section we will give a summary so the later parts of this study can be understood.

3.1 SAMP

The algorithm made by Rumpf and Kaul in [RK21] was made to solve the *social access maximization problem* (SAMP). The formulation of the problem is very general as to be applicable to many situations.

Let L be the set of considered transitlines, where a transitline $l \in L$ is a series of stops with a given traveltime between each stop. A schedule y is a vector with $y_l \in \mathbb{N}$ the amount of vehicles assigned to line l in a given time period. The vehicles are distributed evenly over the time period, so 5 vehicles in an hour gives 1 trip every 12 minutes.

The initial schedule is denoted as y^* . The model has upper and lower bounds $y_l^{\min} \leq y_l \leq y_l^{\max}$ for all $l \in L$. These can be set to 0 and infinity if no restriction is needed, or both to y_l^* if no change is wanted.

Let x be the user flow vector for the transit network, x^* the initial flow vector. User flow represents how busy parts of the network are, which is influenced by the schedule y and found by the $\text{TransitAssignment}(y)$ function, which will be explained further in Section 3.3.

Other restraints of the model are given by operating- and user costs with upper bounds $B_{operator}, B_{user}$ respectively. Operating costs are mostly made up by monetary costs of running the transit network and monetary gain of users paying to use the service. User costs are made up of difficulty in traversing the network, be that in monetary cost or time spent in the system. More on these costs in Sections 3.4.

Let $\text{Access}(y)$ be a function describing the social access objective. This value only depends on the design of the network y and not on user flow x , because the model is designed for a small group of individuals following assumption 1. Different choices for $\text{Access}(y)$ are given in 3.5.

3.2 Network structure

The model uses a representation of the transit network to calculate traveltime and routechoices. For this, a directed graph $G = (V, A)$ is used. V is the set of all relevant nodes, be that origin-, destination-, transitstopnodes. A is the set of arrows linking the nodes together, these represent different modes of travel like walking, bus- or trainriding.

The node set is partitioned in subsets, each containing specific nodes. Let $V_{stop} \subset V$ contain nodes representing the physical location of transitstops. Let $V_{board} \subset V$ be the set of boarding nodes. Each stopnode $i \in V_{stop}$ has boardingnodes $j \in V_{board}$ for every line $l \in L$ that stops at that stop. Boarding nodes are used to distinguish between transitlines. Let $V^+, V^- \subset V$ be origin- and destination nodes, respectively. These represent the physical places where trips begin and end. To calculate social access, subsets $\tilde{V}^+, \tilde{V}^- \subset V$ are added, representing community and facility nodes, respectively.

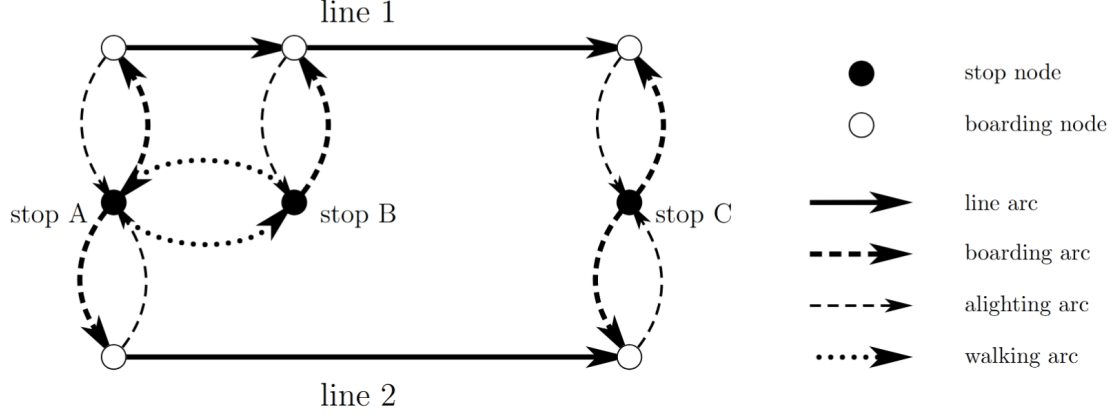


Figure 3.1: Example of network representation containing two lines and three stops. Each stop has nodes in V_{stop} . Line 1 stops at all stops and thus has three boarding nodes, line 2 only has boarding nodes at stop A and stop C. Boarding nodes are connected to their stops with boarding arcs and alighting arcs. Boarding nodes are connected by line arcs if a transitline connects their respective stops. Stop A and stop B are connected by walking arc, because they are close enough. Figure taken from [RK21].

Just like the node set, the arc set A is partitioned in subsets. Let $A_{line} \subset A$ be the arcs that represent the journey made by transitlines and let $A_{board}, A_{alight} \subset A$ be the arcs representing entering or leaving a transit vehicle at a certain stop. Let $A_{walking} \subset A$ be walking arcs, connecting different nodes if their physical distance is walkable.

3.3 Transit Assignment

The transit assignment model, $\text{TransitAssignment}(y)$, models the way daily users interact with a given schedule y . It assumes users try to take their optimal route, which may not be the fastest due to overcrowdedness. The flow produced by the model is called *user-optimal flow* and is often less optimal than when all flow is controlled directly.

There are many such models, but Rumpf and Kaul choose the nonlinear model by Spiess and Florian in [SF89]. This model increases arc cost based on crowdedness, discouraging flows from exceeding line capacity. The user-optimal flow output can not be changed in a way that individual users benefit from changes in only their route. The model is relatively simple, a desired property, as the model is run many times during the algorithm.

3.4 Costs

Generally, operator costs are monetary costs relevant to running the transit system. Things like purchasing and maintaining vehicles, employing drivers and other staff and other costs. In this study, we restrict the total number of busses to not increase. This gives no reason to purchase new vehicles or employ more drivers, so we assume that operator costs will not increase with a new schedule. This means that we can omit the operator cost constraint from the model.

User costs are time costs made by using the system. Things like travel time, wait time, number of transfers and discomfort. All the user costs can be weighted based on transfer mode, 5 minutes of walking may weigh more than 5 minutes of busriding for example. For each $(i, j) \in A$ a flow amount x_{ij} and cost c_{ij} can be implemented. Flow amount represents the amount of users taking that arc, while cost represents some penalty in using that arc, like time needed to traverse it. The user cost function takes the form

$$\text{UserCost}(x, \omega) = \theta_1 \sum_{ij \in A_{line}} c_{ij} x_{ij} + \theta_2 \sum_{ij \in A_{walk}} c_{ij} x_{ij} + \theta_3 \omega,$$

where $\omega \in \mathbb{R}$ is the total time spent waiting at busstops by all users combined.

Due to assumption 1 we do not want the total user cost to increase too much, so we introduce a parameter ε to give a constant upper bound

$$\text{UserCost}(x, \omega) \leq (1 + \varepsilon) \text{UserCost}(x^*, \omega^*).$$

When choosing ε , a balance needs to be found between a large increase, giving a larger set of possible solutions and possibly a more optimal one at the cost of daily user experience.

3.5 Access

The objective function $\text{Access}(y)$ should describe how easy it is to travel from a general community to a general facility, with a focus on equity of access. To do this, a metric is made to measure the accessibility for a specific community $i \in \tilde{V}^+$.

The chosen metric $A_i(y)$ is *gravity-based*, meaning that the attractiveness of facilities reduces gradually as distance increases. An alternative is a discrete distance based approach, where facilities $j \in \tilde{V}^-$ are considered only if they are within a certain distance from the source community i . This approach is not preferred, as facilities that lie just inside the considered radius are weighted equally to facilities that are nextdoor, and facilities that lie just a bit further are not considered at all. This is solved by including a distance function $d_{ij}(y)$, measuring traveltime within G from i to j , and a gravity decay parameter $\beta > 0$. A larger β will result in a higher penalty for further travel.

As the users can choose between facilities, a competition-related metric $F_j(y)$ is used to find the crowdedness of facilities. It is defined as

$$F_j(y) = \sum_{k \in \tilde{V}^+} P_k d_{kj}^{-\beta}(y) \quad \forall j \in \tilde{V}^-$$

where P_k is the population of community k and $d_{kj}^{-\beta}(y)$ is the previously mentioned gravity decay. This crowdedness can be compared to the quality S_j of a facility, usually taken as some metric of patient capacity.

The accessibility metric for a given region is then defined as

$$A_i(y) = \sum_{j \in \tilde{V}^-} \frac{S_j d_{ij}^{-\beta}(y)}{F_j(y)} \quad \forall i \in \tilde{V}^+.$$

This metric was first developed in [Wei76] to describe differential access to employment opportunities, but can also be used for other competition based access cases.

Since the goal is to create more equity in accessibility, not all communities should be included in the overall objective function. Instead, the communities with the worst accessibility should be improved. By maximizing the minimum communities, a more equitable scenario is achieved. The model parameter $\mathcal{K}$ is introduced to achieve this. The objective function is then defined as

$$\text{Access}(y) = \sum_{\substack{\mathcal{K} \text{ minimum elements of} \\ \{A_i(y) | i \in \tilde{V}^+\}}} A_i(y).$$

Note that the objective function will be evaluated many times, and that the $\mathcal{K}$ worst communities are dependent on the y of the current iteration, so the evaluated and improved communities changes while the algorithm is running.

The parameter $\mathcal{K}$ needs to be chosen between 1 and $|\tilde{V}^+|$, where extreme values are not beneficial to the outcome. Low values for $\mathcal{K}$, like 1, may result in minor improvements to a single community with no regard for the relatively large effect on other communities. High values, like $|\tilde{V}^+|$, may result in making large improvements to the highly accessible communities at a cost of making the less accessible communities even worse, as long as the total accessibility increases.

3.6 The model

The SAMP model takes the shape of the following nonlinear program

$$\max_y \text{Access}(y) \quad (3.1)$$

$$\text{s.t.} \quad \sum_{l \in L} y_l \leq \sum_{l \in L} y_l^* \quad (3.2)$$

$$y_l^{\min} \leq y_l \leq y_l^{\max} \quad \forall l \in L \quad (3.3)$$

$$y_l \in \mathbb{N} \quad \forall l \in L \quad (3.4)$$

$$x = \text{TransitAssignment}(y) \quad (3.5)$$

$$\text{UserCost}(x, \omega) \leq (1 + \epsilon) \text{UserCost}(x^*, \omega^*) \quad (3.6)$$

Where (3.1) is the objective function described in section 3.5, (3.2) constrains the total number of vehicles to not increase from the initial amount, (3.3) ensures upper and lower bounds for vehicle allocation, (3.4) ensures a whole number of vehicles is allocated to each line, (3.5) defines user flow as discussed in section 3.3 and (3.6) gives an upper bound to the increase in user cost.

3.7 General algorithm

The SAMP model can be solved heuristically with the following algorithm. It is a combination of *Tabu search* (TS) and *Simulated annealing* (SA), two versions of neighborhood search mentioned in sections 2.2.1 and 2.2.2. Full pseudocode can be found in appendix A.

Using the current transitschedule as a starting point, a neighborhood search is conducted, where neighbors of a solution s are defined as valid solutions that differ only from s by removing one vehicle from a transitline, and adding that vehicle to another line.

During each iteration, the two best neighbors are determined. The best neighbor is chosen as the solution for the next iteration, unless the SA process forces the second best option to be chosen. The option that is not chosen gets put in *long term memory* to be revisited later, while the adding/removing vehicle moves are put in *short term memory* to not be repeated soon, according to TS.

The chance that suboptimal neighbors are chosen is determined by a temperature parameter T , that decreases while the algorithm is running. This ensures more erratic exploration at the start of the algorithm and more exploitation of good solutions towards the end.

After a predetermined amount of iterations is completed, all solutions in the long term memory and their neighbors are revisited. The most optimal solution that is found is then locally optimal.

Chapter 4

Applied model

In their paper, Rumpf and Kaul applied their model to the transit system of Chicago [RK21]. This section will first go over their choices for parameters and then discuss our implementation of the same model on Leiden. It will also go over the different choices of parameters and motivate these choices.

4.1 Chicago

In their application, a transit stop set V_{stop} with 997 different stops was used. These stops were also used as origin-destination (OD) pairs for day-to-day travel volume. Community nodes $\tilde{V}^+$ were defined using the $|\tilde{V}^+| = 77$ Chicago community areas, where node position was taken as population-weighted centroids, based on 2010 census data. Facility nodes $\tilde{V}^-$ were the primary care facilities, of which $|\tilde{V}^-| = 119$ are within the Chicago area. Each facility was given a quality of $S_j = 1$, $\forall j \in \tilde{V}^-$. Walking arcs were created when different locations were 0.75 miles (approximately 1.2 km) within each other, walking speed was set at 4 ft/sec (approximately 4.4 km/h).

Transit lines were acquired from General Transit Feed Specification (GTFS) data. It includes 126 bus lines and 8 train lines. It was also used to estimate the current number of vehicles y_l^* for each line $l \in L$. The time horizon τ was taken as 24-hour period. It was assumed a single type of bus was used, with capacity $b_l = 39$ for all buslines, and a single type of train with capacity $b_l = 228$ for all trainlines. Fleet bounds $y_l^{\min}, y_l^{\max}$ were set to equal y_l^* for all train lines to force these to remain the same. For bus lines $y_l^{\min}$ was set to an amount to achieve at least one arrival per 30 minutes, no maximum $y_l^{\max}$ was set.

Their computational trials also used express lines, existing bus lines only servicing approximately 20% of their initial stops. With less stops, a lower circuit completion time τ^l could be achieved, thus increasing their effective frequency f_l . The stops that remained were in highly accessible communities or in badly accessible communities, so the lines could connect badly accessible communities to facilities. These express lines had initial fleet sizes of $y_l^* = 0$.

The algorithm was run for 500 iterations, followed by a neighborhood search of the best found solution to ensure local optimality. The access function 3.5 was evaluated with gravitational decay parameter $\beta = 1.0$ and community count of $\mathcal{K} = 8$ (approximately 10.39% of 77 communities). The user cost function 3.4 used equal mode weights $\theta_1 = \theta_2 = \theta_3 = 1$ and maximum relative increase $\varepsilon = 0.01$.

They also ran trials where either user cost constraints were ignored, only allowing vehicle swaps between lines and their express lines, and excluding express lines.

4.2 Leiden

Our application concerns the municipality of Leiden, which is much smaller than the Chicago area. This naturally gives a lower amount of busstops, buslines, health facilities and so forth. Although there are multiple train stations in Leiden, the train system is not taken into account for this study.

4.2.1 Transit system

As Qbuzz, the company running the bus system in Leiden, operates in a greater region, most of their buslines lie partially or completely outside of the study area. For this study, only buslines that travel, at least partially, through the study area are considered. Buslines that enter or leave the study area during their route are truncated at their first or last stop within it, apart from one busline that has a single stop outside the municipality before entering the municipality again. Some buslines take equal routes within Leiden, but different ones outside it. These buslines appear identically in our study, apart from their name.

Since Leiden is much smaller than the Chicago area, the implementation of express lines was excluded.

After set of buslines L was selected, a schedule was retrieved from [Qbu25]. The chosen schedule was for monday the 10th of february of 2025, between 14:00 and 15:00 in the afternoon. This weekday and timeframe was chosen as a ordinary, non-rush hour time, that represents ordinary bus travel.

The information given by a schedule is average traveltime between stops, wait time at a stop and amount of busses per line per hour. The algorithm used takes busses per line per timeframe and averages this out to arrive at the frequency of a line, where more busses per timeframe would mean a higher frequency of service.

Names of- and traveltime between busstops can be retrieved from the schedule, giving a transitstop set V_{stop} of 101 different busstops. As most bus stops consist of a pair of physical busstops, one for each direction, the midpoint was taken to represent the physical pair in the graph. Distances to other points, like other busstops or community centers, can be determined and walking arcs can be connected accordingly.

For simplicity it is assumed that the operator uses a single type of bus, the Iveco Crossway EV 13m, with a capacity of $b_l = 50$ for all lines $l \in L$.

4.2.2 Facility and Community nodes

Facility nodes $\tilde{V}^-$ are defined by the general practitioners (GP) offices of Leiden. Taken from [Ned25] a total amount of $|\tilde{V}^-| = 37$ are used. The amount of full-time employed doctors that work at a facility j is taken as the quality S_j of that facility.

For this study, we compare two different methods of defining community nodes $\tilde{V}^+$. The first method concerns itself with postcoderegions in the Netherlands. Each address in the Netherlands has a postcode, consisting of four numbers and two letters. If one only uses the four numbers, the Netherlands can be divided into pc4 regions. There are $|\tilde{V}_{\text{pc4}}^+| = 16$ different pc4 regions within the municipality. Center node location was taken as their middlepoint and each node was assigned population data taken from [All25].

These pc4 regions are still quite large and all, non-day to day trips are simulated to start at the center node to then walk to a bus stop. This may imply that busstops far from the middlepoint of a region are used less then in reality. To remedy this, another way of defining community nodes is found.

According to [Wan22], travelers prefer to take a bus at the nearest bus stop, even when that involves a transfer instead of a direct route. To apply this effect on our study, the regions we choose should reflect trips that start at the closest bus stop. Voronoi diagrams, as defined in section 2.1, are the ideal method of dividing a plane in subsections where each subsection contains one weightpoint and all points that have that weightpoint as the nearest weightpoint.

Using this method, with busstops as weightpoints, we can divide the municipality in $|\tilde{V}_{\text{vor}}^-| = 100$ regions. We have one region less than the number of busstops, because one busstop lies outside the municipality. We estimate the population data for the voronoi regions by looking at the overlap between each voronoi region and the pc4 regions, and assuming the known population of a pc4 region is distributed evenly over its surface. Let

- $\tilde{V}_{\text{vor}}^-$ set of voronoi regions,
- $\tilde{V}_{\text{pc4}}^+$ set of postcode regions,
- s_v surface area for $v \in \tilde{V}_{\text{vor}}^-$,
- d_p population density for $p \in \tilde{V}_{\text{pc4}}^+$,
- $f(v, p)$ fraction of v that lies in p .

Then, the population of a voronoi region v is given by

$$s_v \cdot \sum_{p \in \tilde{V}_{\text{pc4}}^+} d_p f(v, p).$$

4.2.3 Chosen parameters

Since our study tries to improve accessibility for less mobile people, we want to limit the distances walked. Therefore the choice was made to connect nodes via walking arcs if they are within 250 meters of each other, and walking speed was set at 4km/h. Since all day-to-day travel is simulated to begin and end at a busstop, this change only affects those travelers when walking between busstops for a transfer. It will have a much larger effect the group of less mobile people that the algorithm is made to optimize for.

Fleet bounds are set identically as in [RK21], that being no upper bound $y_l^{\max}$ and lower bound $y_l^{\min}$ such that at least two arrivals are made per hour, for all lines $l \in L$. Gravitational decay parameter $\beta = 1.0$, maximum relative user cost increase $\varepsilon = 0.01$ and mode weights $\theta_1 = \theta_2 = \theta_3 = 1$ are also kept the same as in the Chicago case.

Seeing as the amount of pc4 communities is quite small ($|\tilde{V}_{\text{pc4}}^+| = 16$), and two of these regions are very sparsely populated with no bus service, taking roughly 10% of this would not suffice. Thus the community count is set higher at $\mathcal{K}_{\text{pc4}} = 4$, or 25%. The voronoi regions do not have these issues, so their community count can be set at $\mathcal{K}_{\text{vor}} = 10$ to be in line with the Chicago case. **These parameters may be finetuned after running, just like in the paper by Rumpf**

4.3 Explanation of experiment

4.4 Results

4.5 Discussion

Appendix A

Pseudocode

The pseudocode for the solution algorithm discussed in section 3.7, directly taken from [RK21].

x represents the entirety of the flow vector. The vector $e_l \in \{0, 1\}^{|L|}$ represents a vector with a 1 in coordinate l and 0 elsewhere. Sets $\tilde{N}^+, \tilde{N}^-, \tilde{N}^\times$ represent the ADD, DROP and SWAP neighbors kept as candidates after the first pass of the neighborhood search, respectively, while $N^+, N^-, N^\times$ are the second-pass candidates. Bounds $\tilde{N}_{\max}, N_{\max}$ are the required numbers of candidates to be gathered during each pass. $Q_{\text{in}}^{\max}, Q_{\text{out}}^{\max}$ are the inner and outer nonimprovement counter bounds, respectively.

```

1: // INITIALIZATION
2:    $y^{(0)} \leftarrow y^*$  ▷ current real-world fleet sizes
3:    $y_{\text{best}} \leftarrow y^{(0)}$  ▷ best known solution and objective
4:    $\text{BestAccess} \leftarrow \text{Access}(y^{(0)})$  ▷ best known objective
5:    $\text{STM} \leftarrow \emptyset$  ▷ tabu move list, equipped with tabu tenures
6:    $\text{LTM} \leftarrow \emptyset$  ▷ attractive solution set
7:    $T \leftarrow T_0$  ▷ simulated annealing temperature
8:    $t \leftarrow t_0$  ▷ tabu tenure
9:    $Q_{\text{in}} \leftarrow 0$  ▷ inner nonimprovement counter
10:   $Q_{\text{out}} \leftarrow 0$  ▷ outer nonimprovement counter
11:  for  $k = 1, \dots, K$  do ▷ search for a predetermined number  $K$  of local moves
12:    // NEIGHBORHOOD SEARCH
13:     $\tilde{N}^+ \leftarrow \emptyset$  ▷ tentative ADD neighborhood
14:    repeat ▷ first pass (ADD)
15:       $l \leftarrow$  random element of  $L$  not yet considered during this loop
16:      if  $y^{(k)} + e_l$  satisfies vehicle limit bounds (3.1 - 3.3) then ▷ consider only design-feasible
17:        moves
18:        if  $e_l \notin \text{STM}$  or  $\text{Access}(y^{(k)} + e_l) > \text{BestAccess}$  then
19:           $\tilde{N}^+ \leftarrow \tilde{N}^+ \cup \{l\}$  ▷ keep non-tabu candidates (unless improved best)
20:        until  $|\tilde{N}^+| \geq \tilde{N}_{\max}$  or all lines in  $L$  have been considered
21:       $N^+ \leftarrow \emptyset$  ▷ final ADD neighborhood
22:      repeat
23:         $l \leftarrow$  unprocessed element of  $\tilde{N}^+$  with maximum  $\text{Access}(y^{(k)} + e_l)$ 
24:         $x^{(k)} \leftarrow \text{TransitAssignment}(y^{(k)} + e_l)$  ▷ calculate user flows for given move
25:        if  $(y^{(k)} + e_l, x^{(k)})$  satisfy operator and user cost constraints (3.5 - 3.6) then
26:           $N^+ \leftarrow N^+ \cup \{l\}$  ▷ keep only feasible candidates
27:        until  $|N^+| \geq N_{\max}$  or all lines in  $\tilde{N}^+$  have been processed
28:       $\tilde{N}^- \leftarrow \emptyset$  ▷ tentative DROP neighborhood
29:      repeat ▷ first pass (DROP)
30:         $l \leftarrow$  random element of  $L$  not yet considered during this loop
31:        if  $y^{(k)} - e_l$  satisfies vehicle limit bounds (3.1 - 3.3) then ▷ consider only design-feasible
32:          moves
33:          if  $-e_l \notin \text{STM}$  or  $\text{Access}(y^{(k)} - e_l) > \text{BestAccess}$  then
34:             $\tilde{N}^- \leftarrow \tilde{N}^- \cup \{l\}$  ▷ keep non-tabu candidates (unless improved best)
35:          until  $|\tilde{N}^-| \geq \tilde{N}_{\max}$  or all lines in  $L$  have been considered
36:         $N^- \leftarrow \emptyset$  ▷ final DROP neighborhood

```

```

35: repeat
36:    $l \leftarrow$  unprocessed element of  $\tilde{N}^-$  with maximum  $\text{Access}(y^{(k)} - e_l)$ 
37:    $x^{(k)} \leftarrow \text{TransitAssignment}(y^{(k)} - e_l)$   $\triangleright$  calculate user flows for given move
38:   if  $(y^{(k)} - e_l, x^{(k)})$  satisfy operator and user cost constraints (3.5 - 3.6) then
39:      $N^- \leftarrow N^- \cup \{l\}$   $\triangleright$  keep only feasible candidates
40:   until  $|N^-| \geq N_{\max}$  or all lines in  $\tilde{N}^-$  have been processed
41:   if  $|N^+| = |N^-| = 0$  then  $\triangleright$  handle the event of a failed search
42:     if bounds  $N_{\max}$  were used in the previous neighborhood search then
43:       restart neighborhood search while ignoring the bounds  $\tilde{N}_{\max}$ 
44:     else
45:        $Q_{\text{out}} \leftarrow Q_{\text{out}} + 1$   $\triangleright$  work towards diversification
46:       delete from STM the tabu rule with the shortest remaining tenure
47:       restart neighborhood search
48:    $\tilde{N}^\times \leftarrow \{(m, l) \in N^- \times N^+ : z_m = z_l\}$   $\triangleright$  set of all compatible DROP/ADD pairs
49:    $N^\times \leftarrow \emptyset$   $\triangleright$  final SWAP neighborhood
50:   repeat
51:      $(m, l) \leftarrow$  unprocessed element of  $N^\times$  with maximum  $\text{Access}(y^{(k)} - e_m) + \text{Access}(y^{(k)} + e_l)$ 
52:      $x^{(k)} \leftarrow \text{TransitAssignment}(y^{(k)} - e_m + e_l)$ 
53:     if  $y^{(k)} - e_m + e_l, x^{(k)}$  satisfy operator and user cost constraints (3.5 - 3.6) then
54:        $N^\times \leftarrow N^\times \cup \{(m, l)\}$   $\triangleright$  keep only feasible candidates
55:   until  $|N^\times| \geq N_{\max}$  or all moves in  $\tilde{N}^\times$  have been processed
56:    $\tilde{y}^{(k,1)} \leftarrow$  element of  $N^+ \cup N^- \cup N^{\text{times}}$  with greatest access  $\triangleright$  returning two best neighbors
57:    $\tilde{y}^{(k,2)} \leftarrow$  element of  $N^+ \cup N^- \cup N^{\text{times}}$  with second-greatest access
58:   // NEIGHBOR PROCESSING
59:   if  $\text{Access}(\tilde{y}^{(k,1)}) - \text{Access}(\tilde{y}^{(k)}) > \text{then}$ 
60:      $Q_{\text{out}} \leftarrow 0$   $\triangleright$  improvement iteration
61:      $t \leftarrow t_0$ 
62:      $y^{(k,1)} \leftarrow \tilde{y}^{(k,1)}$   $\triangleright$  move to improving neighbor
63:     add rules with tenure  $t$  to STM to prevent undoing the DROP/ADD moves of  $y^{(k,1)}$ 
64:     if  $\text{Access}(\tilde{y}^{(k,1)})_i \text{ BestAccess}$  then
65:        $y_{\text{best}} \leftarrow \tilde{y}^{(k,1)}$   $\triangleright$  update best known solution
66:        $\text{BestAccess} \leftarrow \text{Access}(\tilde{y}^{(k,1)})$ 
67:   else
68:      $Q_{\text{out}} \leftarrow Q_{\text{out}} + 1$   $\triangleright$  nonimprovement iteration
69:      $Q_{\text{in}} \leftarrow Q_{\text{in}} + 1$ 
70:      $r \sim U[0, 1]$   $\triangleright$  random variable drawn uniformly from  $[0, 1]$ 
71:     if  $r < \exp\{-[\text{Access}(y^{(k)}) - \text{Access}(\tilde{y}^{(k,1)})]/T\}$  then
72:       increment  $t$   $\triangleright$  SA criterion passed
73:        $y^{(k+1)} \leftarrow \tilde{y}^{(k,1)}$   $\triangleright$  move to non-improving neighbor
74:       make undoing the DROP/ADD moves of  $y^{(k+1)}$  tabu with tenure  $t$ 
75:        $Q_{\text{in}} \leftarrow 0$ 
76:        $\text{LTM} \leftarrow \text{LTM} \cup \{\tilde{y}^{(k,2)}\}$   $\triangleright$  keep second-best neighbor as an attractive solution
77:     else
78:        $\text{LTM} \leftarrow \text{LTM} \cup \{\tilde{y}^{(k,1)}\}$   $\triangleright$  keep best neighbor as an attractive solution
79:   // UPKEEP
80:   if  $Q_{\text{in}} \geq Q_{\text{in}}^{\max}$  then
81:      $Q_{\text{in}} \leftarrow 0$   $\triangleright$  attempt to diversify
82:      $Q_{\text{out}} \leftarrow Q_{\text{out}} + 1$ 
83:      $y \leftarrow$  randomly chosen element of LTM
84:      $\text{LTM} \leftarrow \text{LTM} \setminus \{y\}$ 
85:      $y^{(k+1)} \leftarrow y$   $\triangleright$  move to a random attractive solution
86:   increment  $t$ 
87:   if  $Q_{\text{out}} \geq Q_{\text{out}}^{\max}$  then
88:      $t \leftarrow t_0$   $\triangleright$  attempt to intensify
89:   remove moves from STM whose tenures have elapsed
90:   apply cooling schedule to  $T$ 
91: return  $(y_{\text{best}}, \text{BestAccess})$ 

```

After the algorithm terminates and we obtain the best known solution, we conduct an exhaustive local search starting at y_{best} . In each iteration of this local search we consider every possible ADD, DROP, and SWAP move without considering tabu rules, always taking the best neighbor and terminating when no neighbors offer any improvement. The final result is guaranteed to be locally optimal.

Appendix B

Data used

B.1 Busschedule

B.2 Busstops

B.3 General Practitioners

B.4 Communities

B.5 Travel data

Bibliography

- [All25] Allecijfers. May 1, 2025. URL: <https://allecijfers.nl/postcode/>.
- [Møl94] Jesper Møller. *Lectures on Random Voronoi Tessellations*. Springer New York, NY, 1994. DOI: <https://doi.org/10.1007/978-1-4612-2652-9>.
- [Ned25] Patientenfederatie Nederland. May 2, 2025. URL: <https://www.zorgkaartnederland.nl/huisartsenpraktijk/leiden>.
- [Qbu25] Qbuzz. Feb. 3, 2025. URL: <https://zhn.qbuzz.nl/?direction=1&date=2025-02-10>.
- [RK21] Adam Rumpf and Hemanshu Kaul. “A public transit network optimization model for equitable access to social services”. eng. In: (2021).
- [SF89] Heinz Spiess and Michael Florian. “Optimal strategies: A new assignment model for transit networks”. In: *Transportation Research Part B: Methodological* 23.2 (1989), pp. 83–102. ISSN: 0191-2615. DOI: [https://doi.org/10.1016/0191-2615\(89\)90034-9](https://doi.org/10.1016/0191-2615(89)90034-9). URL: <https://www.sciencedirect.com/science/article/pii/0191261589900349>.
- [Swe25] Sweco. Sept. 18, 2025. URL: <https://www.sweco.nl/portfolio/het-gorlaeus-gebouw/>.
- [Wan22] M. Wang F.; Ye. “Optimization Method for Conventional Bus Stop Placement and the Bus Line Network Based on the Voronoi Diagram.” In: *Sustainability* 14.7918 (2022). DOI: <https://doi.org/10.3390/su14137918>.
- [Wei76] Jörgen W. Weibull. “An axiomatic approach to the measurement of accessibility”. In: *Regional Science and Urban Economics* 6.4 (1976), pp. 357–379. ISSN: 0166-0462. DOI: [https://doi.org/10.1016/0166-0462\(76\)90031-4](https://doi.org/10.1016/0166-0462(76)90031-4). URL: <https://www.sciencedirect.com/science/article/pii/0166046276900314>.